Where to Spend Maintenance Bytes in Sharded Vector Search: Summary Refresh, Extra Shard Searches, or Vector Push

Anonymous Authors

***Abstract*—A coordinator routes queries to 2 of 32 shards using 16-center summaries. With B summaries' worth of bytes/epoch, should it refresh summaries, search an extra shard, or push vectors? Recent recall@10 is the returned share of top-10 neighbors from the last two epochs; gaps are percentage points (pp) with 95% CIs. The answer depends on where vectors land. When arrivals join the nearest-centroid shard, full refresh changes recent recall@10 over never-refresh by only $+0.55$ pp $[-0.07,\ +1.12]$ (random topics) and $+2.29$ pp $[+1.47,\ +3.17]$ (new topics). There, at B=4, Q=200 queries/epoch, extra search and vector push beat every refresh rule. With one or eight active shards/epoch, vector push wins on random topics. On new topics, refresh wins with one. With eight, it no longer leads vector push at B=2 once data-change's score messages are charged, has no clear winner at B=4, and loses at B=8. There, at B=4, a 64-query recovered-neighbor probe reaches 0.2775 recent recall@10 versus 0.3135 for data-change. Probing spends 22.0% of B and ranks shards worse than new-vector counts; with scores charged and equal refreshes, data-change leads by $+2.66$ pp $[+1.46,\ +4.00]$. We test $100{,}000$ passages at $\approx$1,000 inserts/epoch and omit latency, schedule-aware routing, and some costs.**

I. Introduction

A distributed vector database stores text passages as fixed-length vectors and returns those most similar to a query. To answer questions about newly added documents, the system must find recent arrivals. We measure this with recent recall@10: the share of a query's true top-10 neighbors from the last $W = 2$ epochs that the system returns. We also report overall recall@10. For each query, the coordinator searches $m = 2$ of $S = 32$ shards. It ranks shards by the largest dot product with a center in each $c = 16$-center summary. Each summary is 24,576 bytes, and each center is a 384-float, 1,536-byte vector. The coordinator then merges the results into a top-10 list.

Shards insert arriving passages locally without moving existing vectors, making stored summaries stale so routing misses recent top-10 neighbors. Refreshing all 32 summaries each epoch (*always-fresh*) sends 786,432 bytes, whereas *never-refresh* sends 0 maintenance bytes. Sending $Q = 200$ queries to $m = 2$ shards (1,656 bytes/search) uses 662,400 bytes. The maintenance budget is B summary equivalents, or $24{,}576 \cdot B$ bytes per epoch. That budget cannot refresh every shard.

At equal budgets $B \in \{2, 4, 8\}$, which works best: refreshing summaries, searching an extra shard on uncertain queries (batch or causal online), or pushing new vectors to the coordinator to scan on every query (1,544 bytes/vector)? Second, among refresh rules, does a 64-query recovered-neighbor probe beat round-robin, data-change, query-popularity, or shard-size?

The answer depends on where new vectors land and on the stream: random-topic (*steady*) or cluster-ordered from held-out clusters (*new-topic*). Section V reports gaps in percentage points (pp), with 95% confidence intervals (CIs) on five-seed paired gaps; other values are descriptive. In centroid placement (M1, M1-online), each arrival joins the shard with the nearest initial centroid, so stale setup summaries already route queries there. Refreshing all summaries every epoch changes recent recall@10 over never refreshing by only $+0.55$ pp $[-0.07, +1.12]$ on steady (not significant) and $+2.29$ pp $[+1.47, +3.17]$ on new-topic. At $B = 4$, $Q = 200$, batch extra search, online extra search, and vector push each beat every refresh rule on both streams. In single-segment placement (M2), all arrivals of epoch e go to shard $e \bmod S$, so every refresh rule refreshes it and matches always-fresh exactly. There, vector push wins at every B on steady, where fresh summaries also lower overall recall@10, and refresh wins at every B on new-topic. In eight-segment placement (M3), arrivals join the active shard with the nearest initial centroid in a rotating group of $G = 8$, so 5 to 8 shards change per epoch, more than $B \in \{2, 4\}$ can refresh. On steady, fresh summaries fail our 5 pp gain gate (always-fresh minus never-refresh) and vector push wins at every B. On new-topic, data-change refresh leads vector push at $B = 2$, has no clear winner at $B = 4$, and loses at $B = 8$. That $B = 2$ lead is fragile: its t-interval over five seeds crosses zero, and it relies on leaving uncharged the 128 bytes/epoch that shards need to report data-change scores. In an exploratory rerun, charging those score bytes removes data-change refresh's $B = 2$ lead over vector push on new-topic (-0.16 pp $[-1.89, +1.49]$). At $B = 4$ there, our 64-query recovered-neighbor probe reaches recent recall@10 of 0.2775 versus 0.3135 for data-change refresh. Mostly the probe picks worse shards: with data-change charged so both refresh 3 summaries, data-change still leads by $+2.66$ pp $[+1.46, +4.00]$, while the extra refresh adds 0.94 pp (probing consumes 22.0% of B). On average in every centroid and eight-segment condition, the probe's estimate correlates less strongly with true top-10 neighbors on lifted-in shards than the unrefreshed new-vector count $N_{\text{new}}(s)$ (Spearman 0.26 vs. 0.63 at B=4 on eight-segment new-topic).

We make three contributions, starting with a deterministic, byte-counted simulator that logs every maintenance and query message against exact ground truth (Section III). Second, we adapt search-impact crawl ordering and source-cooperative

synchronization (Section II) into a 64-query refresh rule. It estimates how many new top-10 neighbors, among unrefreshed vectors, a refreshed summary would recover when that summary lifts the shard into the top m. Third, under a pre-registered plan, two addenda, and an exploratory addendum (Section IV), we test eleven methods in three placements (plus online extra search in two) across five seeds on both streams.

II. Related Work

Budgeted crawl and refresh ordering. Pandey and Olston [1] schedule the refreshing of stored web pages by assigning each page p at time t a priority $P(p,t)$ proportional to expected quality improvement and downloading the B highest-priority pages each time unit. In follow-up work, Pandey and Olston [2] prioritize fetching new pages by search impact, defined as how often a page enters the top K results for workload queries (for example, $K = 10$) and estimated from current top-K workload scores.

Stale source summaries in federated search. Kulkarni and Callan [3] study selective search, which partitions a collection by similarity into topic shards and searches only a few shards per query. When collections change over time, routing summaries become stale [4], [5]. Ipeirotis et al. [4] model content changes with survival analysis under limited network and compute resources to contact databases only when needed so summaries stay acceptably up to date. Shokouhi et al. [5] evaluate forty-four dynamic collections over eight weeks, showing that out-of-date representations degrade accuracy and proposing three update policies: constant update (CU, expanding all representations equally each step, the uniform analogue of round-robin refresh), query-log popularity (QL), and collection size (SS).

Push synchronization and status updates under bandwidth limits. Olston and Widom [6] study best-effort cache synchronization when bandwidth limits prevent exact synchronization, so stale copies are permitted. In their source-cooperation design, each source may wait for several updates to accumulate when bandwidth is limited, locally prioritizes its own modified objects, and refreshes modified objects whose priority exceeds its own estimate of a global refresh threshold T, which the cache regulates with occasional feedback messages. Maatouk et al. [7] introduce the Age of Incorrect Information (AoII), a metric that addresses shortcomings of Age of Information and error penalty functions by extending fresh updates to fresh informative updates, so that the penalty grows the longer the monitor has incorrect information.

Partitioned vector index upkeep under drift. Recent work maintains partitioned vector indexes as data and queries evolve. Xu et al. [8] show that updating one-third of the vectors in a static partition index degrades recall because fixed centroids miss gradual shift and raises tail latency because posting lists grow, and propose SPFresh to split partitions and reassign vectors in nearby partitions. Baranchuk et al. [9] show content drift degrades accuracy without reconstruction and propose DeDrift-Lazy to update centroids from assigned vectors. Mohoney et al. [10] combine local re-clustering in Ada-IVF with a policy based on partition temperature, size deviation, and centroid drift, while Quake [11] splits or merges multi-level partitions using a query-latency cost model. Mageirakos et al. [12] propose CrackIVF, which starts small and progressively expands and adapts partitions as queries arrive. Jose [13] studies when a partitioned approximate nearest-neighbor (ANN) index needs active re-partitioning under drift, showing that whether re-partitioning is needed is governed by the router's top-b probe-budget slack (the number b of cells each query probes, which plays the role of our m across shards). Moderate centroid staleness can therefore be traded against a slightly larger probe budget. Single-index maintenance policies trade local maintenance work against extra partition probes within one index.

Distributed vector search routing and summary of differences. Guo et al. [14] present Manu, where shards correspond to insertion and deletion channels, entities are hashed by primary key, and data enters growing segments that seal at a size or idle-time limit. Wang et al. [15] present Semord, where peers in a decentralized overlay exchange heartbeats with liveness state and compact synopses under a capped per-peer synopsis budget, gate epoch synthesis by hysteresis, and guard cached records with a time-to-live and epoch number. They also report refresh overhead in messages and bytes, and Semord peers contact extra peers via semantic expansion (the decentralized analogue of extra shard search) to compensate for routing error, whereas Semord's coordinator baseline uses fresh routing metadata. We instead study a coordinator-routed store whose summaries go stale, under a per-epoch byte budget B.

Across these areas, we adapt top-K search-impact prioritization [1], [2], source-cooperative local prioritization with coordinator cutoff feedback [6], and AoII-motivated informative updates [7] to 16-center shard summaries. We compare that rule against the constant-update (round-robin), query-log popularity, and collection-size policies of Shokouhi et al. [5] and a change-driven score that, like the schedules of Ipeirotis et al. [4], favors summaries most likely to be out of date, but measures change at the shard instead of predicting it. Finally, we test an analogue of the probe-slack hypothesis of Jose [13] and the extra-peer expansion of Semord [15] across distributed shards by comparing all five summary refresh rules against batch and causal online extra shard searches and coordinator vector push at equal budget B.

III. System Model and Policies

We model a coordinator and $S = 32$ shards over $E = 50$ epochs $e \in \{0, \ldots, E-1\}$. Each epoch, clients send new vectors to their assigned shards without moving existing data, the policy spends up to B summary equivalents ($24{,}576 \cdot B$ bytes; Table I), and the coordinator answers Q test queries (default $Q = 200$). Vectors are 384 unit-norm floats (1,536 bytes) compared by dot product $\langle q, v \rangle$. At setup (epoch -1), the initial passages are partitioned into $S = 32$ shards using 32-cluster spherical k-means (20 iterations) from Faiss [16]. Each shard $s \in \{0, \ldots, S-1\}$ indexes its vectors in a local

TABLE I
PER-EPOCH MESSAGE SIZES (S=32, m=2, Q=200, Q_s=64 QUERIES; RN: RECOVERED-NEIGHBOR; B: BYTES).

Message	Composition	Unit (B)	Epoch Total (B=4)
Shard summary	$16 \times 1{,}536$ B	24,576	98,304 B (4 summaries)
Shard search	1,536 B + 120 B	1,656	662,400 B (400 query searches)
Pushed vector	1,536 B + 8 B ID	1,544	97,272 B (63 vectors)
Sample reply	10×4 B scores	40	5,120 B ($Q_s \times m$)
Score cutoffs	$64 \times (4+4)$ B	512	16,384 B (512 B $\times$ S)
Shard score	1×4 B	4	128 B (4 B $\times$ S)
RN overhead	$5{,}120 + 16{,}384 + 128$	n/a	21,632 B (3 summaries left)

Hierarchical Navigable Small World (HNSW) proximity graph [17] ($M = 16$, construction 200, search 100, single-threaded, RNG seed $r + 1009s$ for run seed $r \in \{42, \ldots, 46\}$).

Summaries and routing. At setup, each shard s runs 16-center spherical k-means (15 iterations) on its initial vectors and sends its *published summary* $\mu_s^{\text{pub}} = \{\mu_{s,1}^{\text{pub}}, \ldots, \mu_{s,c}^{\text{pub}}\}$ ($c = 16$ unit centers, 24,576 bytes) to the coordinator. Each shard tracks unrefreshed new vectors $V_{\text{new}}(s)$ of size $N_{\text{new}}(s) = |V_{\text{new}}(s)|$ and wait age $w(s) \in \mathbb{N}_0$ (starting at 0 and incrementing by 1 each epoch shard s is not refreshed, whether or not it received vectors). Each epoch, any shard with $N_{\text{new}}(s) > 0$ updates its *candidate summary* μ_s^{cand} via 16-center spherical k-means (15 iterations) over all $N(s)$ current vectors on shard s, warm-started from μ_s^{pub} (when $N_{\text{new}}(s) = 0$, $\mu_s^{\text{cand}} = \mu_s^{\text{pub}}$). Our simulator updates μ_s^{cand} on all modified shards each epoch for diagnostics; in deployment, non-probing rules only run k-means on refreshed shards. Refreshing shard s sends μ_s^{cand} (24,576 bytes) and sets $\mu_s^{\text{pub}} \leftarrow \mu_s^{\text{cand}}$, $N_{\text{new}}(s) \leftarrow 0$, and $w(s) \leftarrow 0$. For query q, the coordinator scores shards by $\text{score}_{\text{pub}}(q, s) = \max_{1 \le j \le c} \langle q, \mu_{s,j}^{\text{pub}} \rangle$, sorts them in descending order $s_{(1)}, \ldots, s_{(S)}$ (lowest s breaks ties), and probes the top $m = 2$ shards. Each probed shard returns 10 HNSW candidates (8-byte IDs, 4-byte scores, 120 bytes), and the coordinator merges returned lists (plus buffered vectors under vector push), deduplicates IDs by max score, and returns the top 10 (lowest ID breaks ties).

Shard placement modes. Existing vectors never move between shards; we test three placement rules in epochs $e \in \{0, \ldots, 49\}$. **(1) Centroid placement (M1, M1-online).** Each arriving vector v goes to the shard with the nearest initial setup centroid. **(2) Single-segment placement (M2).** All vectors arriving in epoch e are appended to shard $e \bmod S$, loosely modeling segment-based stores such as Manu [14], where new entities enter a growing segment that is sealed at a size or time limit. In our model, recent data therefore sits apart from the older, centroid-organized shards. Because arrivals append to existing shard $e \bmod S$, each segment also keeps its initial topic cluster (with $E = 50 > S = 32$, shards 0 to 17 receive two epochs of arrivals). Only shard $e \bmod S$ has $N_{\text{new}}(s) > 0$, and every rule refreshes modified shards first ($n_B \ge 1$ at $B \ge 2$, where n_B is the number of summaries sent per epoch, defined below). Every rule therefore refreshes shard $e \bmod S$ (re-sending $n_B - 1$ unmodified $\mu_s^{\text{cand}} = \mu_s^{\text{pub}}$) and ties always-fresh on every query. **(3) Eight-segment placement ($G = 8$, M3).** In epoch e, $G = 8$ shards $\{(G \cdot e + j) \bmod S : j \in \{0, \ldots, G-1\}\}$ are active, and each arrival goes to whichever active shard has the nearest initial setup centroid. Across the 5 seeds (250 epochs/stream; Section IV), new vectors reach all 8 active shards in 494/500 epochs: 250/250 on steady (mean 8.00) and 244/250 on new-topic (mean 7.96, min 5). At $B \in \{2, 4\}$ ($n_B \le 4 < G = 8$), no rule can refresh every active shard. At $B = 8$, the four zero-overhead rules ($n_B = 8 = G$) refresh all active shards and tie always-fresh on every query, whereas recovered-neighbor ($n_B = 7 < 8$) need not tie. Both arrival-time modes assume a schedule-oblivious coordinator routing solely by μ_s^{pub}.

TABLE II
EVALUATED METHODS (S=32, m=2; B: BYTES; B=4 IS 98,304 B).

Policy	Role	Per-Epoch Action (B=4)
Round-robin refresh	Refresh rule	n_B=4 summaries (98,304 B)
Data-change refresh	Refresh rule	n_B=4 summaries (98,304 B)
Recovered-neighbor refresh	Refresh rule	n_B=3 summaries (95,360 B)
Query-popularity refresh	Refresh rule	n_B=4 summaries (98,304 B)
Shard-size refresh	Refresh rule	n_B=4 summaries (98,304 B)
Batch extra search	Equal-byte alt.	59 extra searches (97,704 B)
Online extra search	Equal-byte alt.	$\le$ 59 causal extra searches
Vector push	Equal-byte alt.	63 pushed vectors (97,272 B)
Never refresh	Reference	0 summaries (0 B)
Always fresh	Reference	32 summaries (786,432 B)
Oracle routing	Reference	Top-m shards by true neighbors
All-shard search	Reference	Searches all 32 shards

Compared policies and starvation cap. Table II lists the 12 methods. After probing overhead ($B_{\text{rem}} = 24{,}576 \cdot B$ for 4 rules; $B_{\text{rem}} = 24{,}576 \cdot B - 21{,}632$ for recovered-neighbor), a refresh rule sends $n_B = \min(S, \max(0, \lfloor B_{\text{rem}}/24{,}576 \rfloor))$ summaries/epoch. Because $24{,}576 \cdot B$ is an exact multiple of the summary size (zero slack), any charged overhead, however small, costs at least one summary per epoch. The starvation cap is twice a full refresh cycle, $A = \lceil 2S/n_B \rceil$ ($A = 32, 16, 8$ for $n_B = 2, 4, 8$; $A = 64, 22, 10$ for $n_B = 1, 3, 7$). **(1) Round-robin refresh (RR).** Adapts the constant-update (CU) policy of Shokouhi et al. [5] with cyclic cursor $\rho_{\text{rr}} \in \{0, \ldots, S-1\}$, scanning $(\rho_{\text{rr}} + \text{offset}) \bmod S$ in three passes until n_B shards are chosen (first shards with $N_{\text{new}}(s) > 0$ and $w(s) > A$, next remaining shards with $N_{\text{new}}(s) > 0$, then any remaining shards) and advancing $\rho_{\text{rr}} \leftarrow (\rho_{\text{rr}} + n_B) \bmod S$. Rules 2 to 5 map $\text{raw}(s)$ by order-preserving min-max scaling over the S shards (smallest raw score $r_{\min}$, largest $r_{\max}$) to $\text{scaled}(s) = (\text{raw}(s) - r_{\min})/(r_{\max} - r_{\min}) \in [0, 1]$ (0 if $r_{\max} = r_{\min}$). They rank shards lexicographically by $\mathbb{I}[N_{\text{new}}(s) > 0]$ descending, $\mathbb{I}[w(s) > A]$ descending, $\text{scaled}(s)$ descending, and shard ID s ascending. Because the starvation key is binary, non-starved modified shards with tied $\text{scaled}(s)$ (including unrefreshed shards from prior epochs when $n_B < G$) break ties by lowest s. **(2) Data-change refresh (DC).** Each shard's score is $N_{\text{new}}(s)/N(s)$ times mean cosine distance of $v \in V_{\text{new}}(s)$ to its nearest center in μ_s^{pub}, giving $\text{raw}_{\text{dc}}(s) = \frac{1}{N(s)} \sum_{v \in V_{\text{new}}(s)} \max(0, 1 - \max_{1 \le j \le c} \langle v, \mu_{s,j}^{\text{pub}} \rangle)$ (0 when $N_{\text{new}}(s) = 0$). Each shard computes $\text{raw}_{\text{dc}}(s)$ from its own new vectors, so a deployment would send one 4-byte score per shard to the coordinator ($32 \times 4 = 128$ bytes/epoch, the shard-score message of Table I). Our simulator reads these scores directly from shard state: main runs log and charge no score message, and Exploratory Addendum 3 charges 128 bytes/epoch to B.

(3) Recovered-neighbor refresh (RN). Adapts search-impact crawl ordering [2] and source-cooperative cache synchronization [6] using a fixed set of 64 sample queries, written Q_s (we also use Q_s for its size; all Q_s query vectors are broad-

cast to all shards at setup, 3,145,728 bytes uncharged). First, each epoch the coordinator routes each $q \in Q_s$ to its top m shards under μ^{pub} and fetches their top-10 scores (5,120 bytes). Second, it broadcasts the 64 cutoff pairs $(\tau_{\text{route}}(q), \tau_{10}(q))$ of m-th shard score and 10th merged score to all shards (16,384 bytes). Third, each shard s finds $L(s) \subseteq Q_s$ with a *lift-in flip*, where s is outside the published top m and $\max_j \langle q, \mu^{\text{cand}}_{s,j} \rangle > \tau_{\text{route}}(q)$ (or ties with a lower shard ID). Shard s then counts unrefreshed vectors $v \in V_{\text{new}}(s)$ beating $\tau_{10}(q)$ via $\text{raw}_{\text{rn}}(s) = \sum_{q \in L(s)} \sum_{v \in V_{\text{new}}(s)} \mathbb{I}[\langle q, v \rangle > \tau_{10}(q)]$, and returns its 4-byte score (128 bytes across S shards). It checks only $V_{\text{new}}(s)$, not all $N(s)$ vectors, and ignores neighbors on the displaced m-th shard; drop-out flips, where s leaves the top m under μ^{cand}_s, are logged but not scored. **(4) Query-popularity (QL) and (5) Shard-size refresh (SS).** Adapting QL and SS [5], QL sets $\text{raw}_{\text{ql}}(s)$ to cumulative top-m selections of shard s across past test queries, and SS sets $\text{raw}_{\text{ss}}(s)$ to $N(s)$. Unlike $N_{\text{new}}(s)$, neither score resets on refresh, so both re-refresh the highest-scoring modified shards until other shards exceed $w(s) > A$.

Equal-byte alternatives and references. Three alternatives keep setup summaries frozen and spend up to $24{,}576 \cdot B$ bytes/epoch. **(1) Batch extra search (ES).** Tests an analogue of probe-budget slack [13] and extra-peer expansion [15] by spending up to $N_{\text{extra}} = \lfloor 24{,}576 \cdot B / 1{,}656 \rfloor$ extra shard searches per epoch (59 at $B = 4$), ordering candidate searches $r \in \{m{+}1, \ldots, S\}$ across the epoch's Q queries by rank offset $r - (m+1)$ ascending, routing score gap $\text{score}_{\text{pub}}(q, s_{(m)}) - \text{score}_{\text{pub}}(q, s_{(r)})$ ascending, query index ascending, and $s_{(r)}$ ascending (an optimistic batch allocation). **(2) Online extra search (ESO).** Removing batch look-ahead, an arriving query searches $s_{(m+1)}$ (1,656 bytes) if $\text{score}_{\text{pub}}(q, s_{(m)}) - \text{score}_{\text{pub}}(q, s_{(m+1)}) \le \tau_e$ and budget remains, where τ_e is the N_{extra}-th smallest gap in epoch $e-1$. Epoch 0 sets τ_0 from Q_s (or the max sample gap when $N_{\text{extra}} > 64$ at $B{=}8$); at $Q{=}50, B{=}4$ ($N_{\text{extra}} > Q$), τ_e is the max gap of epoch $e{-}1$ ($\le$ 50 searches/epoch). **(3) Vector push (PV).** This policy pushes up to $N_{\text{push}} = \lfloor 24{,}576 \cdot B / 1{,}544 \rfloor$ new vectors/epoch (63 at $B = 4$) newest-first (the last N_{push} vectors in epoch e's stream array) to the coordinator. The coordinator retains all pushed vectors across all 50 epochs without eviction and scans them by exact dot product on every query. Four unbudgeted references run on the same queries: *never refresh* (0 bytes), *always fresh* (all 32 summaries rebuilt each epoch; a heuristic 16-center reference, not an upper bound), *oracle routing*, and *all-shard search* (all 32 shards). Oracle routing picks the m shards with the most true overall top-10 neighbors/query, bounding m-shard routing on overall recall@10 rather than recent recall@10, $> m$ shards, or vector push.

Byte accounting and uncharged costs. Network latency, queueing, concurrency, and node failures are not modeled; bytes are counted on a deterministic message bus (Table I) where each budgeted method spends at most $24{,}576 \cdot B$ bytes/epoch at $B \in \{2, 4, 8\}$ (default $B = 4$). Six items are uncharged, starting with (1) ingest traffic ($\approx 1{,}000 \times 1{,}536$ bytes/epoch to shards, identical across policies). (2) For recovered-neighbor, amortizing the 3,145,728-byte setup broadcast over 50 epochs (62,915 bytes/epoch) would leave 13,757 bytes at $B = 4$ (no summaries), a negative budget at $B = 2$, and 4 instead of 7 summaries at $B = 8$. Its local shard compute (15-iteration 16-center k-means rebuilds and $Q_s \times m = 128$ local HNSW sample searches/epoch), $(m{+}1)$-th score for drop-out logging, m-th shard ID for tie-breaks, and the per-epoch lists telling each shard which sample queries to answer are also uncharged. (3) Data-change's $32 \times 4 = 128$ score bytes/epoch are uncharged in main runs; under the zero-slack floor, charging them cuts n_B from $2, 4, 8$ to $1, 3, 7$ (Exploratory Addendum 3, Section IV). (4) Every refresh rule uses an uncharged bit $\mathbb{I}[N_{\text{new}}(s) > 0]$ (even a 4-byte bitmap triggers the $\lfloor B_{\text{rem}} / 24{,}576 \rfloor$ floor), shard-size uses $N(s)$, and our rank-correlation baseline uses $N_{\text{new}}(s)$ (free only if the coordinator tracks insert counts). (5) Extra search charges 1,656 bytes/search, but extra local HNSW searches ($\le$ 59/epoch at $B = 4$) and 3-shard fan-out or an extra round trip are uncharged. (6) Vector push assumes global arrival ordering and uses uncharged coordinator memory and scans: up to 3,150 vectors at $B = 4$ and 6,350 at $B = 8$, adding $6.2\times$ to $12.4\times$ the 512 routing dot products/query. A $W \cdot N_{\text{push}} = 126$-vector FIFO buffer at $B = 4$ suffices for $W = 2$ recent recall@10. Not charging items 2 to 4 favors refresh rules, while items 5 and 6 favor the alternatives.

TABLE III

EXPERIMENT SUITES (1,116 RUNS) AND HYPOTHESES (PL.: PLACEMENT; SEG.: SEGMENT; CEN.: CENTROID; M1O: M1-ONLINE; SENS: CAP SENSITIVITY; PR: PREDICTION; GT: GATE; H: HYPOTHESIS; LK: LOOK-AHEAD/UNDERSPEND; RN: RECOVERED-NEIGHBOR; AF: ALWAYS FRESH; H2/H2B/H4/H4B: LOWER 95% CI > 0; H3/H3B: MEAN ≥ 5 PP, LOWER CI > 0; H1/GT2/GT3: MEAN).

ID	Pl.	Setting	Runs / Pass Rule (Recent Recall@10)
M1	Cen.	$B{=}4, Q{\in}\{50,200,800\}$	330 runs (11 meth., 5 seeds, $A{=}\lceil 2S/n_B \rceil$)
H1	Cen.	$B{=}4, Q{=}200$	AF $-$ never refresh < 5 pp (mean)
H2	Cen.	$B{=}4, Q{=}200$	Batch extra & push $>$ best refresh
M1o	Cen.	$B{=}4, Q{\in}\{50,200,800\}$	30 runs (online extra search, 5 seeds)
H2b	Cen.	$B{=}4, Q{=}200$	Online extra & push $>$ best M1 refresh
M2	1-seg.	$B{\in}\{2,4,8\}, Q{=}200$	330 runs (11 meth., 5 seeds, $A{=}\lceil 2S/n_B \rceil$)
PR2	1-seg.	$B{\in}\{2,4,8\}, Q{=}200$	Max per-query $\lvert$rule $-$ always fresh$\rvert = 0$
GT2	1-seg.	$B{=}4, Q{=}200$	Always fresh $-$ never refresh ≥ 5 pp
H3	1-seg.	$B{=}4, Q{=}200$	RN beats other 4 rules by ≥ 5 pp
H4	1-seg.	$B{\in}\{2,4,8\}, Q{=}200$	Always fresh $>$ batch extra & push
Sens	1-seg.	$B{=}4, Q{=}200, A{=}8$	66 runs (11 methods, 3 seeds); ties M2
M3	8-seg.	$B{\in}\{2,4,8\}, Q{=}200$	360 runs (12 methods, 5 seeds, $G{=}8$)
PR3	8-seg.	$B{=}8, Q{=}200$	4 rules ($n_B{=}8{=}G$) tie always fresh (0)
GT3	8-seg.	$B{=}4, Q{=}200$	Always fresh $-$ never refresh ≥ 5 pp
H3b	8-seg.	$B{=}4, Q{=}200$	RN beats other 4 rules by ≥ 5 pp
H4b	8-seg.	$B{\in}\{2,4,8\}, Q{=}200$	Best refresh $>$ online extra & push
LK	Both	M1 (Q), M3 (B)	Batch $-$ online (look-ahead $+$ underspend)

IV. EXPERIMENTAL SETUP

Dataset and arrival streams. We use passages and queries from the MS MARCO dataset [18], where questions come from anonymized user queries and passages are extracted from web documents retrieved by Bing. With fixed data seed 42, we sample 100,000 passages, 1,000 development-set test queries, and $Q_s = 64$ training-set sample queries (excluding training queries in the development split or matching a test query's normalized text; sample-to-test cosine similarity < 0.9999). All passages and queries are embedded as unit-norm 384-float vectors (1,536 bytes) using the six-layer MiniLM model from Sentence-Transformers [19]; each epoch $e \in \{0, \ldots, 49\}$ draws $Q \le 1{,}000$ test queries without replacement (RNG seed $r + 10000 + e$). We test two 50-epoch streams in every

suite. **(1) Steady stream.** At setup, 50,000 passages build the $S = 32$ shards, and the remaining 50,000 are permuted by the run's seed and inserted at 1,000 vectors/epoch. **(2) New-topic stream.** Modeled after the Big-ANN NeurIPS'23 streaming runbook, which clusters points into 64 clusters and inserts them cluster by cluster [20]. Before setup, we run 64-cluster spherical k-means (20 iterations, seeded by the run's seed) [16] over all 100,000 vectors and place the 64 clusters in a seeded random order. The first 32 clusters form the initial set at setup and the other 32 form the stream, ordered by cluster across the 50 epochs (5-seed mean 50,880.6 streamed vectors). Seed 42 has 47,273 initial and 52,727 streamed vectors, inserting 1,055/epoch in epochs 0 to 26 and 1,054 in epochs 27 to 49.

Ground truth, recall, and Spearman correlation (ρ). At setup and in each epoch e, all inserted vectors are added to an exact inner-product index [16] returning each test query's 10 exact ground-truth neighbors and their insertion epochs. Overall recall@10 is the fraction returned in the coordinator's merged top-10 list. Our primary metric is *recent recall@10* ($W = 2$), the fraction of a query's exact top-10 ground-truth neighbors inserted in $[\max(0, e - W + 1), e]$ (epochs e and $e - 1$) returned in the top-10 list. We exclude queries with no recent ground-truth neighbors (counts in Section V), and identical ground truth keeps method comparisons paired. Before a refresh, we hold the other shards' published summaries, μ^{pub}, fixed. A shard's *lift-in neighbor count* is the number of exact top-10 neighbors on shard s among the Q test queries that its candidate summary, μ_s^{cand}, would lift into the top m. This count includes non-recent neighbors. It does not subtract neighbors displaced from the selected shards. A larger Q samples more queries. On recovered-neighbor runs (and counterfactually on other runs), we compute Spearman ρ across the 32 shards between lift-in counts and both $\text{raw}_{\text{rn}}(s)$ (using only $Q_s = 64$, independent of Q) and $N_{\text{new}}(s)$, averaging five per-seed means over epochs where both vary (ρ for data-change was not measured).

Analysis plan and addenda. Before any run, a version-controlled plan fixed parameters, message sizes, sweeps over $S, m, B \le 16, Q \le 400$, and three initial hypotheses (RN beating other rules by ≥ 5 pp in M1, a break-even Q vs. alternatives, and larger refresh gains under cluster arrivals and query shift). Before the pilot, code review added the new-topic stream (a random split covers every cluster), replaced $+0.5w(s)$ aging with the $w(s) > A$ cap, reclassified always-fresh as a reference, and skipped constant-score Spearman epochs. A seed-42 pilot showed small centroid refresh gains ($+0.67$ pp steady, $+1.98$ pp new-topic; setup summaries rank a vector's shard top-2 for 85.5% of new-topic arrivals, first for 64.9%) and heavy $A = 8$ cap forcing (84% recovered-neighbor, 64% data-change on new-topic). Before main runs, Addendum 1 dropped unused sweeps, set $Q \in \{50, 200, 800\}$, added single-segment placement, set $A = \lceil 2S/n_B \rceil$, corrected pushed vectors from 1,536 to 1,544 B (8-B ID), cached ground truth, and replaced the initial hypotheses with H1, H2, GT2, H3, and H4. H1 and H2 restate pilot results, which included extra search and vector push, and reuse seed 42. M1 pass 1 stopped at 149/330 runs after one failed twice (113 hit memory errors; 112 retried OK). Pass 2 ran the remaining 181 from the same commit (47 retried OK); testing the committed analysis script on 117 partial M1 runs printed one value and changed no rule. Code review of a result-free draft found the single-segment tie and batch look-ahead before M2 started. Addendum 2, committed when 140/330 M2 runs had finished (none opened) and before M1-online or M3 started, added eight-segment placement ($G = 8$), online extra search, PR2, PR3, GT3, H3b, H2b, H4b, and LK (Table III). Later edits before main analysis changed no rule, and H2b pairs M1-online with M1 refresh runs on unchanged code. After inspecting all main results, we committed the post hoc Exploratory Addendum 3 before any charged run to charge data-change's 128 score bytes/epoch under the zero-slack floor ($n_B \in \{1, 3, 7\}$, $A \in \{64, 22, 10\}$). It ran 42 simulations against stored baselines: 30 M3 runs ($B \in \{2, 4, 8\}, Q$=200), 10 M1 runs (B=4, Q=800), and 2 replication runs with the charge off that matched stored M3 and M1 runs on every per-query and non-CPU metric. It defined X1 (equal-n_B vs. RN), X2 (vs. PV and ESO), X3 (charged minus uncharged DC), and X4 (M1 Q=800), plus descriptive predictions P1 (X1 new-topic B=4 lower CI > -1.00 pp) and P2 (X2 PV new-topic B=2 lower CI ≤ 0.00 pp), with no pass/fail gates and 64 unadjusted intervals (32 comparisons $\times 2$ metrics).

Rule selection and two-stage bootstrap CIs. In Table III, H1 tests < 5 pp M1 headroom, GT2/GT3 test ≥ 5 pp headroom to gate H3/H3b, and H2/H2b/H4/H4b test equal-byte alternatives. Picking the best refresh rule on the same runs is conservative for H2/H2b (Q=200) and slightly optimistic per B in H4/H4b; Table V fixes the Q=200 winner across Q (not best on steady at $Q \in \{50, 800\}$). Suites M1, M1o, M2, and M3 use seeds 42 to 46 (Sens: 42 to 44; data seed 42, so CIs condition on sampled passages, queries, and rule selection, unadjusted for multiplicity). We compute 95% CIs via a two-stage paired bootstrap (2,000 replicates, seed 42). Stage 1 resamples the five seeds with replacement (shrinking between-seed variance by a factor of 4/5 and using percentile quantiles), and Stage 2 resamples paired query differences within each drawn seed (counting query-level variance a second time). It pools valid queries across drawn seeds (weighting seeds by valid queries, unlike equal-weighted 5-seed t-intervals). Each test query appears 2.5, 10, and 40 times/seed on average at $Q \in \{50, 200, 800\}$ and epoch queries share μ_s^{pub} (both treated as independent in Stage 2 and shared across seeds via the 1,000-query set). Bootstrap CIs are 19% to 27% narrower than t-intervals on the five M3 headline tests (GT3 on both streams, H3b RN minus DC, and H4b refresh minus PV at $B \in \{2, 4\}$ on new-topic), where between-seed exceeds within-seed variance. When between-seed variance is small, they are wider (51% wider for H3b RN minus QL).

V. Experimental Results

In all 5 main suites (1,116 runs, 0 missing) and Exploratory Addendum 3 (42 runs; 22/30 M3 runs retried OK after a first-attempt memory error), every run finished and spent

TABLE IV

HYPOTHESIS AND PREDICTION TESTS AT $Q = 200$ (RECENT RECALL@10 PAIRED GAPS WITH 95% CIS; H: HYPOTHESIS; PR: PREDICTION; GT: GATE; LK: LOOK-AHEAD/UNDERSPEND; REF.: BEST REFRESH; DC: DATA-CHANGE; RN: RECOVERED-NEIGHBOR; ES/ESO: BATCH/ONLINE EXTRA SEARCH; PV: VECTOR PUSH; NR: NEVER REFRESH; AF: ALWAYS FRESH; *LEAD OR TIE WEAKENS OR DISAPPEARS WHEN DATA-CHANGE'S SCORE MESSAGES ARE CHARGED, TABLE VIII). H4/H4B PASS IF REFRESH BEATS BOTH ALTERNATIVES (3-WAY GAPS IN TABLE VI).

Test	Comparison (Q=200)	Steady Stream	New-Topic Stream	Outcome (Steady / New-Topic)
H1	M1 AF − NR (B=4)	+0.55 pp [−0.07, +1.12]	+2.29 pp [+1.47, +3.17]	Pass / Pass (means and upper CIs < 5 pp)
H2	M1 ES − Ref. (B=4)	+2.76 pp [+2.33, +3.22]	+1.97 pp [+0.74, +3.15]	Pass / Pass
	M1 PV − Ref. (B=4)	+1.88 pp [+1.44, +2.36]	+2.09 pp [+1.00, +3.27]	Pass / Pass
H2b	M1 ESO − Ref. (B=4)	+2.55 pp [+2.17, +2.97]	+1.84 pp [+0.58, +3.01]	Pass / Pass
PR2	M2 5 rules − AF (all B)	+0.00 pp [+0.00, +0.00]	+0.00 pp [+0.00, +0.00]	Pass / Pass (exact tie; max diff 0.0000)
GT2	M2 AF − NR (B=4)	+1.66 pp [+1.17, +2.24]	+22.61 pp [+20.83, +24.40]	Fail (< 5 pp) / Pass (≥ 5 pp)
H3	M2 RN − other 4 (B=4)	+0.00 pp [+0.00, +0.00]	+0.00 pp [+0.00, +0.00]	Gated / Fail (tie by construction)
H4	M2 Ref. − PV (B=2)	−1.40 pp [−1.96, −0.82]	+19.40 pp [+17.67, +21.12]	Fail (PV wins) / Pass (Refresh wins)
	M2 Ref. − PV (B=4)	−4.31 pp [−5.09, −3.47]	+16.04 pp [+14.40, +17.73]	Fail (PV wins) / Pass (Refresh wins)
	M2 Ref. − PV (B=8)	−10.15 pp [−11.02, −9.23]	+9.64 pp [+8.04, +11.26]	Fail (PV wins) / Pass (Refresh wins)
PR3	M3 4 rules − AF (B=8)	+0.00 pp [+0.00, +0.00]	+0.00 pp [+0.00, +0.00]	Pass / Pass (exact tie; max diff 0.0000)*
	M3 RN − AF (B=8)	−0.19 pp [−0.64, +0.27]	−1.59 pp [−2.34, −0.98]	Not predicted to tie (means shown; max per-query recall difference 1.0 on single-recent-neighbor queries)
GT3	M3 AF − NR (B=4)	+1.16 pp [+0.38, +1.91]	+6.74 pp [+5.16, +8.16]	Fail (< 5 pp) / Pass (≥ 5 pp)
H3b	M3 RN − DC (B=4)	−0.39 pp [−1.06, +0.32]	−3.60 pp [−4.93, −2.26]	Gated / Fail (data-change wins)*
H4b	M3 Ref. − PV (B=2)	−1.79 pp [−2.38, −1.20]	+1.47 pp [+0.03, +2.90]	Fail (PV wins) / Pass, fragile (refresh ahead)*
	M3 Ref. − PV (B=4)	−3.67 pp [−4.57, −2.85]	+1.04 pp [−0.50, +2.53]	Fail (PV wins) / Fail (no clear winner)*
	M3 Ref. − PV (B=8)	−7.17 pp [−8.00, −6.31]	−3.59 pp [−5.37, −1.86]	Fail (PV wins) / Fail (PV wins)
LK	M1 ES − ESO (B=4)	+0.21 pp [+0.03, +0.37]	+0.13 pp [−0.07, +0.35]	Small at $Q = 200$ (≤ 0.21 pp)
	M3 ES − ESO (B=4)	+0.16 pp [−0.01, +0.32]	+0.30 pp [+0.08, +0.52]	Small at $Q = 200$ (≤ 0.30 pp)

$\leq B$ on charged messages (uncharged costs in Section III). Table IV lists every registered test. At Q=200 (10,000 queries/seed, 50 epochs), recent recall@10 is defined on 11,917/50,000 steady (23.83%) and 6,375/50,000 new-topic queries (12.75%, 1,155 to 1,370/seed; at $Q \in \{50, 800\}$: 3,040, 48,223 steady; 1,629, 25,626 new-topic).

A. Centroid Placement (M1 and M1-Online)

Small fresh-summary headroom (H1). Under centroid placement ($Q = 200, B = 4$), each arrival joins the shard with the nearest initial centroid, so setup summaries already steer queries to new arrivals. Refreshing all 32 summaries (786,432 B/epoch) changes recent recall@10 from 0.7026 (never refresh) to 0.7081 on steady (+0.55 pp [−0.07, +1.12], not significant) and from 0.4643 to 0.4872 on new-topic (+2.29 pp [+1.47, +3.17]). Both upper 95% bounds sit below 5 pp, confirming H1 across seeds 42 to 46 (with pilot seed 42). On out-of-sample seeds 43 to 46, pooled/unweighted means are +0.52 pp and +0.52 pp (−0.47 to +1.31 pp) on steady and +2.37 pp and +2.41 pp (+1.66 to +3.18 pp) on new-topic. Overall recall@10 changes by +0.00 pp [−0.25, +0.23] on steady and +0.58 pp [+0.35, +0.80] on new-topic. Against m=2 oracle routing, which picks the two shards holding the most overall top-10 neighbors (0.7996 steady, 0.6335 new-topic; all-shard 0.9985, 0.9968), full refresh closes only 5.6% and 13.5% of the never-refresh-to-oracle gap.

Equal-byte alternatives at $Q = 200$ (H2 and H2b). Over 50 epochs at B=4 (98,304 B/epoch), four refresh rules spend 4,915,200 B, RN spends 4,768,000 B (7,913,728 amortized), ES spends 4,885,200 B, ESO spends 4,649,386 (steady) and 4,604,342 (new-topic; 95.2% and 94.3% of ES), and PV spends 4,863,600 B (3,150 vectors). Refresh rules span 0.7008 (RR) to 0.7042 (RN, best; none beats NR reliably) on steady and 0.4645 (RR) to 0.4813 (DC, best) on new-topic. At equal bytes, ES (0.7317, 0.5009), ESO (0.7297, 0.4996), and PV (0.7230, 0.5022) all beat best refresh. On steady, their gains over best refresh are +2.76 pp [+2.33, +3.22], +2.55 pp [+2.17, +2.97], and +1.88 pp [+1.44, +2.36]. On new-topic, they are +1.97 pp [+0.74, +3.15], +1.84 pp [+0.58, +3.01], and +2.09 pp [+1.00, +3.27] (H2 and H2b pass; all three beat every refresh rule, and batch extra search and vector push beat always-fresh). On out-of-sample seeds 43 to 46, every per-seed ES and PV gain over best refresh is positive. Probing a third shard on ≤ 59 close-margin queries/epoch (14.8% shard load at Q=200, 3-shard fan-out or extra round trip) improves general routing. It raises overall recall@10 over best refresh by +3.03 pp [+2.83, +3.19] and +2.86 pp [+2.66, +3.03] on steady and +2.25 pp [+2.09, +2.41] and +2.08 pp [+1.91, +2.25] on new-topic. Vector push is a sampling effect: buffering 3,150/50,000 streamed vectors (6.30%, $p = 0.0630$) predicts $R_{\text{NR}} + p(1 - R_{\text{NR}}) = 0.7214$ on steady from never-refresh R_{NR} (observed 0.7230; p falls as inserts grow; summary size is fixed). It raises overall recall@10 over best refresh by only +0.54 pp [+0.38, +0.74] (steady) and +0.32 pp [+0.12, +0.53] (new-topic), scanning $\leq$ 3,150 vectors in coordinator memory (126 FIFO vectors suffice for recent recall@10).

TABLE V

CENTROID QUERY-RATE SWEEP AT $B = 4$ (RECENT RECALL@10 GAPS IN PP, 95% CIS; REF.: BEST Q=200 REFRESH, RECOVERED-NEIGHBOR ON STEADY, DATA-CHANGE ON NEW-TOPIC; ES/ESO: BATCH/ONLINE EXTRA SEARCH; PV: VECTOR PUSH; GAPS ARE REF. − ALT., OPPOSITE TO H2/H2B IN TABLE IV; *LEAD OR TIE WEAKENS OR DISAPPEARS WHEN DATA-CHANGE'S SCORE MESSAGES ARE CHARGED, TABLE VIII).

Q	Comp.	Steady Stream (pp)	New-Topic Stream (pp)
50	Ref. − ES	−8.15 [−9.70, −6.55]	−8.51 [−10.44, −6.66]
	Ref. − ESO	−6.82 [−8.08, −5.51]	−6.52 [−8.74, −4.58]
	Ref. − PV	−1.89 [−2.97, −0.89]	−2.10 [−4.18, −0.13]
	ES − ESO	+1.33 [+0.77, +2.00]	+1.99 [+1.12, +3.01]
200	Ref. − ES	−2.76 [−3.22, −2.33]	−1.97 [−3.15, −0.74]
	Ref. − ESO	−2.55 [−2.97, −2.17]	−1.84 [−3.01, −0.58]
	Ref. − PV	−1.88 [−2.36, −1.44]	−2.09 [−3.27, −1.00]
	ES − ESO	+0.21 [+0.03, +0.37]	+0.13 [−0.07, +0.35]
800	Ref. − ES	−0.73 [−0.96, −0.48]	+1.01 [+0.60, +1.42]*
	Ref. − ESO	−0.69 [−0.93, −0.44]	+1.06 [+0.65, +1.48]*
	Ref. − PV	−1.78 [−2.15, −1.43]	−1.76 [−2.18, −1.34]
	ES − ESO	+0.03 [+0.00, +0.06]	+0.05 [+0.01, +0.09]

Query-rate sweep ($Q \in \{50, 200, 800\}$) and look-ahead. Table V varies Q at $B = 4$, holding the $Q = 200$ refresh winner fixed across Q (unlike Table VI, which picks the best rule per B). On steady, recovered-neighbor (0.7163) trails data-change (0.7226) by +0.63 pp [+0.02, +1.29] at Q=50 (DC minus PV: −1.26 pp [−2.33, −0.19]). At Q=800, the five rules span only 0.6996 (RN) to 0.7003 (DC). Extra search

TABLE VI

ARRIVAL-TIME EQUAL-BYTE COMPARISONS (Q=200, RECENT RECALL@10 GAPS IN PP, 95% CIS; REF.: BEST REFRESH PER B, M2 5 TIE AF, M3 SS AT B=2 STEADY, DC AT B=4 STEADY AND $B \leq 4$ NEW-TOPIC, 4 TIE AF AT B=8; ES/ESO: BATCH/ONLINE EXTRA SEARCH; PV: VECTOR PUSH; SS: SHARD-SIZE; DC: DATA-CHANGE; AF: ALWAYS FRESH; *LEAD OR TIE WEAKENS OR DISAPPEARS WHEN DATA-CHANGE'S SCORE MESSAGES ARE CHARGED, TABLE VIII).

ID (B)	Comp.	Steady Stream (pp)	New-Topic Stream (pp)
M2 (2)	Ref. − ES	+1.26 [+0.79, +1.76]	+21.93 [+20.16, +23.62]
	Ref. − PV	−1.40 [−1.96, −0.82]	+19.40 [+17.67, +21.12]
	ES − PV	−2.66 [−3.00, −2.32]	−2.53 [−3.28, −1.85]
M2 (4)	Ref. − ES	+0.87 [+0.33, +1.46]	+21.44 [+19.59, +23.22]
	Ref. − PV	−4.31 [−5.09, −3.47]	+16.04 [+14.40, +17.73]
	ES − PV	−5.18 [−5.76, −4.65]	−5.40 [−6.11, −4.70]
M2 (8)	Ref. − ES	−0.03 [−0.59, +0.65]	+20.33 [+18.33, +22.26]
	Ref. − PV	−10.15 [−11.02, −9.23]	+9.64 [+8.04, +11.26]
	ES − PV	−10.13 [−10.80, −9.47]	−10.69 [−11.76, −9.67]
M3 (2)	Ref. − ESO	−1.08 [−1.54, −0.62]	+2.75 [+0.97, +4.33]*
	Ref. − PV	−1.79 [−2.38, −1.20]	+1.47 [+0.03, +2.90]*
	ESO − PV	−0.71 [−1.12, −0.29]	−1.28 [−2.26, −0.43]
M3 (4)	Ref. − ESO	−2.09 [−2.84, −1.33]	+3.86 [+2.10, +5.31]*
	Ref. − PV	−3.67 [−4.57, −2.85]	+1.04 [−0.50, +2.53]*
	ESO − PV	−1.58 [−2.24, −0.91]	−2.82 [−3.79, −1.93]
M3 (8)	Ref. − ESO	−4.28 [−5.17, −3.43]	+1.47 [+0.07, +2.81]
	Ref. − PV	−7.17 [−8.00, −6.31]	−3.59 [−5.37, −1.86]
	ESO − PV	−2.89 [−3.79, −2.05]	−5.05 [−6.47, −3.79]

and vector push beat every refresh rule at every Q on steady (smallest gap, DC minus ESO at Q=800: −0.63 pp [−0.88, −0.36]). Summary refresh and pushed vectors amortize across all queries, whereas $B = 4$ buys at most 59 extra searches per epoch (59.0%, 14.8%, and 3.7% shard load at $Q \in \{50, 200, 800\}$). On new-topic at $Q = 800$ with score bytes uncharged ($n_B = 4$), data-change refresh (0.4872) overtakes batch (0.4771, by +1.01 pp [+0.60, +1.42]) and online extra search (0.4766, by +1.06 pp [+0.65, +1.48]). In Exploratory Addendum 3 (X4, n_B=3, Table VIII), charged data-change edges batch extra search on new-topic by only +0.38 pp [+0.01, +0.74] (t-interval [+0.13, +0.64] pp) and trails both extra searches on steady. That lead is fragile. Its lower CI bound is +0.01 pp (touching zero under other bootstrap seeds), it comes from epochs 0 to 29 (+0.68 pp vs. −0.20 pp in epochs 30 to 49), and it is not ahead on overall recall@10 (−0.20 pp [−0.38, +0.02]). Vector push (0.5048 at Q=800) beats refresh on new-topic at every Q: uncharged refresh minus vector push is −2.10 pp [−4.18, −0.13] at Q=50 (t-interval [−4.11, −0.21] pp) and −1.76 pp [−2.18, −1.34] at Q=800 (charged: Table VIII). At $Q \geq 200$, LK (ES − ESO in Table V) is at most +0.21 pp [+0.03, +0.37] (Q=200 steady, t-interval [−0.00, +0.42] pp, seed 45 −0.04 pp); on new-topic, ESO spends 94.3% to 96.9% of ES bytes, mixing look-ahead with underspend. At Q=50, LK (+1.33 pp [+0.77, +2.00] steady, +1.99 pp [+1.12, +3.01] new-topic) mixes look-ahead with per-query and byte caps. Online, by design, makes ≤ 1 search/query (50/epoch) and spends 83.1% and 83.0% of batch bytes.

B. Arrival-Time Segment Placement (M2 and M3)

Single-segment placement (M2: PR2, GT2, H3, H4). Epoch e writes only to shard $e \bmod 32$, which every rule refreshes at $B \geq 2$, matching AF in all 30 conditions (PR2: +0.00 pp [+0.00, +0.00], max diff 0.0000; Table IV). Re-sending $n_B - 1$ unmodified summaries ($\mu_s^{\text{cand}} = \mu_s^{\text{pub}}$) consumes 50%, 75%, and 87.5% of B for the four zero-overhead

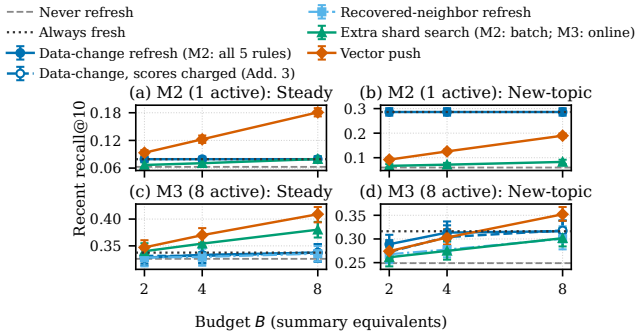


Fig. 1. Recent recall@10 across $B \in \{2, 4, 8\}$ at Q=200 in M2 (top, batch ES; all 5 equal AF) and M3 (bottom, online ESO); y-axes differ. Dashed blue (c, d): data-change with its 128 B/epoch of score messages charged ($n_B \in \{1, 3, 7\}$; exploratory Addendum 3, M3 only). Error bars: unpaired per-method 95% CIs, overlapping even where paired gaps in Table VI exclude zero (Panel c, B=2: SS is +0.12 pp [−0.47, +0.77] above plotted DC).

TABLE VII

REFRESH RULES AND BASELINES (Q=200, RECENT RECALL@10; ST.: STEADY; NT: NEW-TOPIC; NR: NEVER REFRESH; RR: ROUND-ROBIN; DC: DATA-CHANGE; RN: RECOVERED-NEIGHBOR; QL: QUERY-POPULARITY; SS: SHARD-SIZE; AF: ALWAYS FRESH; M2 OMITTED: ALL 5 TIE AF; DC IS THE UNCHARGED RULE; CHARGED GAPS ARE IN TABLE VIII).

Cond.	Str.	NR	RR	DC	RN	QL	SS	AF
M1 (B=4)	St.	0.7026	0.7008	0.7040	0.7042	0.7025	0.7030	0.7081
M3 (B=2)	St.	0.3258	0.3272	0.3280	0.3283	0.3291	0.3292	0.3374
M3 (B=4)	St.	0.3258	0.3262	0.3333	0.3293	0.3307	0.3299	0.3374
M3 (B=8)	St.	0.3258	0.3374	0.3374	0.3355	0.3374	0.3374	0.3374
M1 (B=4)	NT	0.4643	0.4645	0.4813	0.4729	0.4661	0.4661	0.4872
M3 (B=2)	NT	0.2489	0.2451	0.2889	0.2677	0.2555	0.2666	0.3163
M3 (B=4)	NT	0.2489	0.2537	0.3135	0.2775	0.2726	0.2823	0.3163
M3 (B=8)	NT	0.2489	0.3163	0.3163	0.3003	0.3163	0.3163	0.3163

TABLE VIII

EXPLORATORY ADDENDUM 3: CHARGED DATA-CHANGE MINUS EACH METHOD (RECENT RECALL@10 GAPS IN PP, 95% CIS; CHARGED DC SENDS $n_B \in \{1, 3, 7\}$ SUMMARIES AND LEAVES 24,448 B/EPOCH OF B UNSPENT; X1: VS. RN AT EQUAL n_B; X2: VS. PV AND ESO; X3: VS. UNCHARGED DC; X4: M1, Q=800, B=4; ES/ESO: BATCH/ONLINE EXTRA SEARCH; PV: VECTOR PUSH; †FRAGILE, SEE TEXT).

Set (B)	Minus	Steady Stream (pp)	New-Topic Stream (pp)
M3 (2)	X1 RN	+0.30 [−0.29, +0.88]	+0.48 [−0.48, +1.66]
	X2 PV	−1.58 [−2.11, −1.06]	−0.16 [−1.89, +1.49]
	X2 ESO	−0.87 [−1.41, −0.36]	+1.12 [−0.39, +2.62]
	X3 DC	+0.33 [−0.39, +1.08]	−1.63 [−2.90, −0.48]†
M3 (4)	X1 RN	+0.19 [−0.29, +0.63]	+2.66 [+1.46, +4.00]
	X2 PV	−3.87 [−4.72, −3.11]	+0.10 [−1.40, +1.52]
	X2 ESO	−2.29 [−2.90, −1.68]	+2.92 [+1.39, +4.26]
	X3 DC	−0.20 [−0.71, +0.30]	−0.94 [−1.55, −0.32]
M3 (8)	X1 RN	+0.19 [−0.29, +0.64]	+1.76 [+1.04, +2.59]
	X2 PV	−7.16 [−7.98, −6.33]	−3.42 [−5.15, −1.75]
	X2 ESO	−4.28 [−5.17, −3.42]	+1.63 [+0.27, +2.95]†
	X3 DC	+0.00 [−0.17, +0.19]	+0.16 [−0.10, +0.41]
M1 (4)	X4 ES	−0.49 [−0.78, −0.16]	+0.38 [+0.01, +0.74]†
	X4 ESO	−0.45 [−0.75, −0.12]	+0.43 [+0.07, +0.79]
	X4 PV	−1.54 [−1.96, −1.12]	−2.39 [−2.84, −1.99]
	X4 DC	+0.17 [−0.01, +0.35]	−0.63 [−0.89, −0.38]

rules; a deployment could save those bytes or mix in vector push or extra search (not tested). Every pairwise refresh gap in H3 is +0.00 pp [+0.00, +0.00] (failing H3 by construction), and the $A = 8$ sensitivity runs (seeds 42 to 44) match M2 on every query (3-seed means 0.0810 steady, 0.2799 new-topic). The value of refreshing that active shard splits by stream (Fig. 1a,b, Table VI). On steady, 1,000 random vectors from all topics land on shard $e \bmod 32$ each epoch, and never-refresh gets 0.0627 recent recall@10, matching $m/S = 2/32 = 0.0625$ chance (all-shard 0.9996; oracle by overall neighbors 0.2346). Plausibly because 16 centers cannot represent a uni-

form topic mix, always-fresh reaches only 0.0793 (+1.66 pp [+1.17, +2.24], failing GT2) and changes overall recall@10 from 0.5072 to 0.4914 (−1.59 pp [−1.74, −1.45], negative in every seed). Vector push therefore wins on steady at every B (H4, Table VI): pushing 3.10%, 6.30%, and 12.70% of new vectors at $B \in \{2, 4, 8\}$ gives 0.0933, 0.1224, and 0.1808 ($R_{\text{NR}} + p(1 - R_{\text{NR}}) = 0.0917$, 0.1217, and 0.1817). On new-topic, arrivals come from one or two held-out clusters that 16 centers fit cleanly: always-fresh jumps from 0.0600 to 0.2861 (+22.61 pp [+20.83, +24.40], passing GT2; oracle 0.4442, all-shard 0.9963) and raises overall recall@10 from 0.5097 to 0.6084 (+9.87 pp [+8.87, +10.77]). Across $B \in \{2, 4, 8\}$ on new-topic (H4, Table VI), fresh summaries beat batch extra search and vector push on recent recall@10, narrowing as B pushes more vectors while 1 summary suffices. On overall recall@10, they beat vector push by +8.97 pp [+8.02, +9.85], +8.04 pp [+7.04, +8.96], and +6.12 pp [+5.14, +7.02].

Eight-segment placement ($G = 8$, M3: GT3, H3b, PR3, H4b) and Exploratory Addendum 3 (X1 to X3). With $G = 8$ active shards per epoch, steady arrivals mix topics on each active shard: never-refresh is 0.3258 and always-fresh is 0.3374 (oracle 0.5046, all-shard 0.9982). Full refresh fails GT3 on steady (+1.16 pp [+0.38, +1.91]) and changes overall recall@10 by −0.91 pp [−1.02, −0.79] (negative in all five seeds). On new-topic, GT3 passes in aggregate, raising recent recall@10 from 0.2489 to 0.3163 (+6.74 pp [+5.16, +8.16], overall +1.73 pp [+1.12, +2.33]; oracle 0.4863, all-shard 0.9967), though seed 42 is +4.10 pp (< 5 pp) and the t-interval is [+4.79, +8.75] pp (unweighted mean +6.77 pp). On new-topic at $B = 4$ (Table VII), H3b fails: recovered-neighbor (0.2775) loses to data-change (0.3135) by −3.60 pp [−4.93, −2.26] (overall −0.42 pp [−0.71, −0.10], t-interval [−0.90, +0.07] pp). Recovered-neighbor does not differ reliably from query-popularity (0.2726, +0.50 pp [−0.22, +1.14], positive in all 5 seeds, t-interval [+0.03, +0.93] pp) or shard-size (0.2823, −0.48 pp [−1.79, +0.62]), and beats only round-robin (0.2537, +2.38 pp [+1.27, +3.34]). Round-robin's cursor advances $n_B \in \{2, 4\}$ shards/epoch while the active group advances G=8, often refreshing shards whose new vectors are no longer recent and doing no better than never-refresh (−0.37 pp [−1.12, +0.33] at B=2, +0.48 pp [−0.50, +1.39] at B=4). Exploratory Addendum 3 (Table VIII) shows that recovered-neighbor loses mainly through shard choice. It charges data-change's 128 score B/epoch ($n_B \in \{1, 3, 7\}, A \in \{64, 22, 10\}$, leaving 24,448 B/epoch unspent), so both rules refresh equally often, while RN still spends 21,504 B/epoch more plus its uncharged 3,145,728-B setup broadcast. At equal n_B on new-topic (X1), charged data-change beats recovered-neighbor at $B = 4$ (P1; t-interval [+0.91, +4.57] pp; overall +0.42 pp [+0.11, +0.69], t-interval crossing zero) and $B = 8$, with all 10 seed gaps positive, but not reliably at $B = 2$ or on steady. By linearity, the 3.60 pp H3b gap at B=4 splits into 2.66 pp from shard choice at n_B=3 (74%) and 0.94 pp from the extra summary (at B=2: 2.11 pp splits into 0.48 pp and 1.63 pp, whose t-interval crosses zero). That extra summary exists because, in main runs, probing uses 21,632 B/epoch (44.0%, 22.0%, 11.0% of $B \in \{2, 4, 8\}$; modified-only cutoffs would not raise n_B), leaving $n_B \in \{1, 3, 7\}$, while data-change's score bytes are uncharged ($n_B \in \{2, 4, 8\}$). At n_B=3 on new-topic (B=4), RN had 100 cap-forced refreshes across 750 versus 16 for charged DC. In the 62/250 all-zero probe epochs (ties broken by shard ID), the X1 gap averaged +5.42 pp (1,428 queries) versus +1.86 pp across the other 188 epochs (4,947 queries). At $B = 8$ (PR3), the four uncharged $n_B = 8$ rules match always-fresh on every query (+0.00 pp [+0.00, +0.00], max diff 0.0000), whereas charged data-change ($n_B = 7$) no longer ties always-fresh (X3, Table VIII). Recovered-neighbor ($n_B = 7$) differs from always-fresh at $B = 8$ by −0.19 pp [−0.64, +0.27] on steady and −1.59 pp [−2.34, −0.98] on new-topic (max per-query recall difference 1.0000, that is, 100 pp, on single-recent-neighbor queries).

Across $B \in \{2, 4, 8\}$ in eight-segment placement (H4b, Fig. 1c,d, Table VI), vector push wins at every B on steady (0.3471, 0.3700, and 0.4090, matching $R_{\text{NR}} + p(1 - R_{\text{NR}}) = 0.3467$, 0.3683, and 0.4114). On new-topic, vector push beats never refresh by about 0.2 to 1.0 pp more than uniform sampling predicts, plausibly because it takes the newest tail of each cluster-ordered epoch batch (not tested). On new-topic with score bytes uncharged, data-change (0.2889) beats online extra search (0.2613) and leads vector push (0.2742, overall +1.03 pp [+0.63, +1.43]) at $B = 2$. At $B = 4$, it beats online extra search with no clear winner against vector push (+1.04 pp [−0.50, +2.53]; overall +0.33 pp [−0.29, +0.97]). At $B = 8$ on new-topic, refresh leads online extra search by +1.47 pp [+0.07, +2.81] (t-interval [−0.12, +3.07] pp crosses zero) while losing to vector push (0.3521) on both metrics (−3.59 pp [−5.37, −1.86]; overall −1.13 pp [−1.66, −0.61]). That overall gain scans ≤ 6,350 retained vectors, whereas a $W \cdot N_{\text{push}}$ FIFO buffer suffices for recent recall. The uncharged $B = 2$ lead over vector push (+1.47 pp [+0.03, +2.90]) is fragile: its lower CI bound is +0.03 pp, seed 42 is −0.29 pp, and the t-interval ([−0.35, +3.25] pp) crosses zero. In Exploratory Addendum 3 (X2, Table VIII), charging data-change's 128 score bytes/epoch cuts it from 2 to 1 summary at $B = 2$ and removes that lead. Charged data-change minus vector push is −0.16 pp [−1.89, +1.49] (P2; t-interval [−2.56, +2.18] pp), although it is ahead on overall recall@10 (+0.62 pp [+0.35, +0.90]). At $B = 4$ there is no clear winner, and vector push leads at $B = 8$ on new-topic and at every B on steady. Against online extra search on new-topic, charged data-change does not lead reliably at B=2, leads at B=4, and leads fragilely at B=8 (t-interval [+0.10, +3.17] pp, seed 42 at −0.17 pp). It trails on overall recall@10 at B=4 (−0.79 pp [−1.26, −0.25]; t-interval crosses zero) and B=8 (−3.43 pp [−3.77, −3.02]). Batch ES adds 7.2%, 14.8%, and 29.5% extra HNSW load at $B \in \{2, 4, 8\}$. Its LK gaps are −0.01 pp [−0.11, +0.10], +0.16 pp [−0.01, +0.32], and +0.18 pp [+0.03, +0.33] on steady and +0.24 pp [+0.07, +0.40], +0.30 pp [+0.08, +0.52], and +0.28 pp [+0.09, +0.48] on new-topic. ESO spends 90.8% to 96.8% of ES bytes (88.7% to 89.5% of refresh at B=2). Under the ES slope across

TABLE IX
MEAN SPEARMAN ρ WITH LIFT-IN COUNTS AND CAP SHARES (250 EPOCHS/COND.; ST.: STEADY; NT: NEW-TOPIC; DEF.: DEFINED-ρ EPOCHS, MEANS ON DEF.; RN-CONST.: EPOCHS (% OF 250) WITH 64-QUERY ESTIMATE ALL ZERO WHILE LIFT-IN COUNTS VARY; CAP: $w(s)>A$ RN SHARE).

Cond.	Str.	ρ(RN)	$\rho(N_{\text{new}})$	Def., RN-Const. (%)	Cap
M1 (Q=50)	St.	0.04	0.14	71, 145 (58.0%)	27.2%
M1 (Q=200)	St.	0.03	0.20	83, 167 (66.8%)	27.2%
M1 (Q=800)	St.	0.03	0.32	83, 167 (66.8%)	27.2%
M1 (Q=50)	NT	0.12	0.30	157, 89 (35.6%)	21.7%
M1 (Q=200)	NT	0.18	0.50	160, 90 (36.0%)	21.7%
M1 (Q=800)	NT	0.21	0.67	160, 90 (36.0%)	21.7%
M3 (B=2)	St.	0.07	0.41	200, 50 (20.0%)	0.0%
M3 (B=4)	St.	0.10	0.47	129, 121 (48.4%)	17.2%
M3 (B=8)	St.	0.14	0.47	64, 185 (74.0%)	5.0%
M3 (B=2)	NT	0.25	0.65	242, 8 (3.2%)	0.0%
M3 (B=4)	NT	0.26	0.63	188, 62 (24.8%)	13.3%
M3 (B=8)	NT	0.42	0.63	168, 81 (32.4%)	5.3%

budgets, its unspent searches are worth about 0.13 to 0.18 pp, changing no H4b winner but confounding LK.

C. Rank Correlation, Starvation Cap, and Threats

Spearman rank correlation and starvation cap. Table IX compares mean Spearman ρ across 32 shards between lift-in counts and $\text{raw}_{\text{rn}}(s)$ vs. $N_{\text{new}}(s)$ on RN runs (ρ for DC not measured; $N_{\text{new}}(s)$ not tested as a rule). In M1, RN's refreshes, estimate, and Cap are independent of Q (Q_s=64), whereas lift-in counts depend on Q and are all zero in 34 (steady) and 4 (new-topic) epochs at Q=50 and 1 and 1 epochs of M3 B=8 (outside Def. and RN-Const.). In every centroid and eight-segment condition, mean ρ(RN) is below $\rho(N_{\text{new}})$ (means without CIs over epochs where $\text{raw}_{\text{rn}}(s)$ varies, favoring the probe). Counterfactually on data-change runs at Q=200, B=4, ρ(RN) vs. $\rho(N_{\text{new}})$ is 0.03 vs. 0.16 and 0.14 vs. 0.31 in M1, and 0.11 vs. 0.41 and 0.34 vs. 0.50 in M3. By contrast, $\text{raw}_{\text{rn}}(s) = 0$ across all 32 shards in every RN-Const. epoch of Table IX, giving zero ranking signal from 21,632 probe bytes and falling back to $w(s) > A$ and ascending shard ID s. In M2, ρ is 1.00 in every defined epoch (1 modified shard/epoch). Because $\text{raw}_{\text{ql}}(s)$ and $\text{raw}_{\text{ss}}(s)$ do not reset on refresh, $w(s) > A$ forces 27.9% to 28.0% (steady) and 26.9% to 27.3% (new-topic) of QL and SS refreshes at Q=200, B=4 in M1 (vs. 0.0% and 6.9% for DC).

Threats to validity. First, Section III's six uncharged costs affect policies differently (RN's 3,145,728-B setup broadcast and local compute, ES HNSW load, coordinator vector scans, and DC's 128 score B/epoch in main runs). Exploratory Addendum 3 charged DC's score bytes ($n_B \in \{1, 3, 7\}$), removing data-change's B=8 PR3 tie (+0.16 pp [−0.10, +0.41] vs. always-fresh) and B=2 H4b lead (−0.16 pp [−1.89, +1.49] vs. vector push) and leaving a fragile Q=800 recent-recall lead over batch extra search (+0.38 pp [+0.01, +0.74]). Even so, charged data-change beat recovered-neighbor at n_B=3 by +2.66 pp [+1.46, +4.00] on new-topic despite recovered-neighbor spending 21,504 bytes/epoch more plus its uncharged setup broadcast. Because the zero-slack floor leaves 24,448 bytes/epoch unspent for charged data-change (and 2,944 bytes/epoch for recovered-neighbor at B=4), this charge is a pessimistic integer-packing case; we did not run a carryover variant, under which 128 bytes/epoch totals < 1 summary over 50 epochs. Second, in M2 and M3, arrivals append to shards keeping initial topic clusters; on steady, rebuilding 16-center summaries over mixed shards changes overall recall@10 by −1.59 pp [−1.74, −1.45] and −0.91 pp [−1.02, −0.79] (AF minus NR), plausibly shifting centers toward random arrivals (not tested; per-segment summaries not modeled). Both placements also assume a schedule-oblivious coordinator routing only by μ_s^{pub}; routing directly to known active shards ($e \bmod S$ or the G active shards scored by setup centroids) costs zero maintenance bytes (not tested). Third, at $\approx$ 1,000 inserts/epoch, summary size is fixed while the arrival share buffered by PV (3.10%, 6.30%, 12.70% on steady) falls as inserts grow, which may favor refresh at higher ingest rates (not tested). Fourth, all runs use one embedding model (384-float MiniLM on 100,000 MS MARCO passages, data seed 42), fixed S=32, m=2, c=16, and 5 seeds (42 to 46; 3 in Sens).

VI. CONCLUSION

Where to spend a per-epoch budget B to recover recent top-10 neighbors depends on vector placement and topic drift. Under centroid placement, setup summaries already guide queries to shards receiving new vectors, so full refresh every epoch changes recent recall@10 over never refreshing by only +0.55 pp [−0.07, +1.12] on steady and +2.29 pp [+1.47, +3.17] on new-topic. At $Q = 200, B = 4$, causal online extra search and vector push beat every refresh rule on both streams. Data-change refresh overtakes extra search only on new-topic at $Q = 800$, by 1.01 pp uncharged and 0.38 pp with its score bytes charged (fragile, not ahead on overall recall@10), while trailing vector push. Under arrival-time placement, 16-center summaries fail to summarize random steady arrivals and change overall recall@10 by −1.59 pp [−1.74, −1.45] and −0.91 pp [−1.02, −0.79] when refreshed, leaving vector push ahead at every B. On new-topic arrivals, refreshing active summaries raises recent recall@10 by +22.61 pp [+20.83, +24.40] in single-segment placement and +6.74 pp [+5.16, +8.16] in eight-segment placement, beating vector push across $B \in \{2, 4, 8\}$ in single-segment placement. In eight-segment placement on new-topic, refresh has no clear winner against vector push at $B = 4$ and loses at $B = 8$. At $B = 2$ it edges vector push only with score bytes uncharged; charging them costs a whole summary under the zero-slack floor and leaves no lead (−0.16 pp [−1.89, +1.49]). There, at $B = 4$, recovered-neighbor loses to data-change by −3.60 pp [−4.93, −2.26] with score bytes uncharged, mainly because the probe picks worse shards. With data-change's 128 score bytes/epoch charged so both get 3 refreshes, data-change still leads by +2.66 pp [+1.46, +4.00]; the other 0.94 pp comes from the one refresh that probing (22.0% of B) takes away. The probe's estimate correlates less strongly with lift-in neighbor counts than $N_{\text{new}}(s)$ on average in every centroid and eight-segment condition.

For coordinator-routed vector stores in our tested regime ($S = 32, m = 2, c = 16$, $\approx$ 1,000 inserts/epoch), these results suggest three rules accounting for uncharged costs. (1) Under centroid placement at $Q \leq 200$, spend bytes on

causal online extra search if shards can absorb up to 59 more HNSW searches/epoch (50 at $Q = 50$) and 3-shard fan-out (or an extra round trip). Alternatively, spend them on vector push if the coordinator can buffer and scan recent vectors (3,150 vectors at $B = 4$). Reserve summary refresh for higher query rates under topic drift, where data-change refresh overtook extra search at $Q = 800$ (fragilely once its score bytes are charged) but trailed vector push. (2) Under arrival-time placement without schedule-aware routing, refresh the active shard summary when arrivals cluster by topic into one segment. On new-topic with $G = 8$ active segments, fresh summaries and vector push are close at $B \leq 4$ and vector push leads at $B = 8$. Avoid rebuilding 16-center k-means summaries on shards mixing all topics uniformly; there, vector push recovers recent neighbors while leaving the setup summaries, and so routing, unchanged. (3) When scheduling refreshes across active shards, rank shards by the data-change score rather than paying 21,632 bytes/epoch (plus setup broadcast) for 64-query cutoff probing. With its 128 bytes/epoch charged, data-change still beat the probe at equal refreshes on new-topic at $B \in \{4, 8\}$. The new-vector count $N_{\text{new}}(s)$ tracked lift-in neighbor counts better than the probe in every centroid and eight-segment condition, though it was not tested as a refresh rule. Because summary size is fixed while PV's buffered arrival share at fixed B falls as inserts grow, higher ingest rates may favor refresh (not tested). Beyond our single-process simulator on 100,000 MS MARCO MiniLM vectors, future work should test adaptive summaries, schedule-aware routing, higher ingest rates, and tail latency.

ACKNOWLEDGMENT

Gemini (Google) drafted the simulator, analysis code, figure and number scripts, and the text of every section. Claude (Anthropic) checked each citation against its source, recomputed reported numbers, and wrote or edited text in every section. Gemini, Claude, and Codex (OpenAI) reviewed drafts; Codex edited no files. Result numbers come from scripts run on stored run outputs. The authors take responsibility for the content.

REFERENCES

[1] S. Pandey and C. Olston, "User-centric web crawling," in *Proceedings of the 14th International Conference on World Wide Web (WWW '05)*, 2005, pp. 401–411. [Online]. Available: https://doi.org/10.1145/1060745.1060805

[2] ——, "Crawl ordering by search impact," in *Proceedings of the International Conference on Web Search and Web Data Mining (WSDM '08)*, 2008, pp. 3–14. [Online]. Available: https://doi.org/10.1145/1341531.1341535

[3] A. Kulkarni and J. Callan, "Selective search: Efficient and effective search of large textual collections," *ACM Transactions on Information Systems*, vol. 33, no. 4, pp. 1–33, 2015. [Online]. Available: https://doi.org/10.1145/2738035

[4] P. G. Ipeirotis, A. Ntoulas, J. Cho, and L. Gravano, "Modeling and managing content changes in text databases," in *21st International Conference on Data Engineering (ICDE '05)*, 2005, pp. 606–617. [Online]. Available: https://doi.org/10.1109/ICDE.2005.91

[5] M. Shokouhi, M. Baillie, and L. Azzopardi, "Updating collection representations for federated search," in *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '07)*, Amsterdam, The Netherlands, 2007, pp. 511–518. [Online]. Available: https://eprints.gla.ac.uk/3860/1/shokouhi3860.pdf

[6] C. Olston and J. Widom, "Best-effort cache synchronization with source cooperation," in *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data (SIGMOD '02)*, 2002, pp. 73–84. [Online]. Available: https://doi.org/10.1145/564691.564701

[7] A. Maatouk, S. Kriouile, M. Assaad, and A. Ephremides, "The Age of Incorrect Information: A new performance metric for status updates," *IEEE/ACM Transactions on Networking*, vol. 28, no. 5, pp. 2215–2228, 2020. [Online]. Available: https://arxiv.org/abs/1907.06604

[8] Y. Xu *et al.*, "SPFresh: Incremental in-place update for billion-scale vector search," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 2023, pp. 545–561. [Online]. Available: https://arxiv.org/abs/2410.14452

[9] D. Baranchuk, M. Douze, Y. Upadhyay, and I. Z. Yalniz, "DeDrift: Robust similarity search under content drift," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 10 992–11 001. [Online]. Available: https://arxiv.org/abs/2308.02752

[10] J. Mohoney *et al.*, "Incremental IVF index maintenance for streaming vector search," arXiv preprint arXiv:2411.00970, 2024. [Online]. Available: https://arxiv.org/abs/2411.00970

[11] ——, "Quake: Adaptive indexing for vector search," in *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*, 2025, pp. 153–169, arXiv preprint arXiv:2506.03437. [Online]. Available: https://www.usenix.org/system/files/osdi25-mohoney.pdf

[12] V. Mageirakos, B. Wu, and G. Alonso, "Cracking vector search indexes," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 3951–3964, 2025. [Online]. Available: https://doi.org/10.14778/3749646.3749666

[13] J. Jose, "When does a partitioned ANN index need active re-partitioning under drift? A characterization and benchmark," Research Square preprint, version 1, 2026. [Online]. Available: https://doi.org/10.21203/rs.3.rs-9916583/v1

[14] R. Guo *et al.*, "Manu: A cloud native vector database management system," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3548–3561, 2022. [Online]. Available: https://doi.org/10.14778/3554821.3554843

[15] S. Wang, Y. Liu, Y. Hua, X. Zhang, and C. Qian, "Semord: Learned semantic-preserving placement and low-fanout routing for distributed vector search," arXiv preprint arXiv:2609.25514, 2026. [Online]. Available: https://arxiv.org/abs/2609.25514

[16] M. Douze *et al.*, "The Faiss library," arXiv preprint arXiv:2401.08281, 2024. [Online]. Available: https://arxiv.org/abs/2401.08281

[17] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. [Online]. Available: https://arxiv.org/abs/1603.09320

[18] T. Nguyen *et al.*, "MS MARCO: A human generated MAchine Reading COmprehension dataset," in *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016)*, ser. CEUR Workshop Proceedings, vol. 1773, 2016, arXiv preprint arXiv:1611.09268. [Online]. Available: https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf

[19] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992. [Online]. Available: https://doi.org/10.18653/v1/D19-1410

[20] H. V. Simhadri *et al.*, "Results of the Big ANN: NeurIPS'23 competition," arXiv preprint arXiv:2409.17424, 2024. [Online]. Available: https://arxiv.org/abs/2409.17424